

Greedy heuristics for the Power Dominating Set (PDS) problem

Sadia Tabassum
Department of CSE, BUET
1905091@ugrad.cse.buet.ac.bd

Md Azizul Haque Nadim
Department of CSE, BUET
1905059@ugrad.cse.buet.ac.bd

Abu Wasif
Department of CSE, BUET
wasif.cse.buet.ac.bd

Abstract—The Power Dominating Set (PDS) problem asks for a minimum set of vertices that observe an entire graph under a domination rule and a propagation (zero-forcing) rule. It models the placement of phasor measurement units (PMUs) for full observability of an electrical grid. We present a family of greedy construction heuristics for PDS, paired with a post-processing pruning stage, and evaluate them on the BCSPWR power-network benchmark series. Our central finding is that post-processing pruning consistently reduces solution size across all cases. Among the constructors, the effective-degree based greedy introduces the least redundancy and produces the best results both before and after pruning. We further compare a naive single-pass pruner with a fort-based pruner that reaches the same minimal sizes at reduced computational cost on the largest grids.

Index Terms—power dominating set, zero forcing, greedy heuristics, PMU placement, graph algorithms

I. INTRODUCTION

Wide-area monitoring of electrical grids relies on phasor measurement units (PMUs), whose placement must guarantee that the state of every bus is observable [1]. This requirement is captured by the Power Dominating Set (PDS) problem [2], a generalization of the dominating set problem in which observation spreads through a propagation rule in addition to direct domination. Formally, a selected vertex observes itself and all of its neighbors (the domination rule), and an observed vertex with exactly one unobserved neighbor forces that neighbor to become observed (the propagation, or zero-forcing, rule). A set S is a power dominating set if repeatedly applying these rules until no new vertex can be observed results in all vertices being monitored. PDS is NP-hard [3], so practical placement on large grids depends on heuristics. Although exact exponential-time algorithms [4] and more recent efficient methods [5] have been proposed, heuristic approaches remain essential for scaling to large power networks.

We study greedy construction heuristics that build a PDS incrementally, selecting one vertex at a time by a scoring rule until the whole graph is observed [6]. Because such construction is monotone—a placed vertex is never reconsidered—the resulting set is valid but typically not minimal. We therefore pair construction with a pruning stage that removes redundant vertices, and we compare two pruning strategies: a naive single-pass remover and a fort-based pruner that avoids unnecessary propagation checks.

II. MAIN RESULTS

A. Heuristics

All heuristics share a common observation engine that applies the domination and propagation rules until no new vertex is observed, running in $O(m)$ time on a graph with m edges.

We evaluate five greedy constructors. **H1 (Degree based greedy)** selects the highest static-degree vertex. **H2 (Effective-degree-based greedy)** selects the vertex with the most currently *unobserved* neighbors, recomputed at each step. **H3 (Multi-hop greedy)** scores vertices by a weighted count of neighbors at distances one, two, and three. **H4 (Effective multi-hop greedy)** restricts the multi-hop score to unobserved vertices. **H5 (Near-twin-guided greedy)** is a two-phase method that first selects near-twin vertices—adjacent vertices with near-identical neighborhoods—by degree, then completes the solution with degree-based greedy.

After construction, each vertex is tested for removal: if the set still observes the whole graph without it, the vertex is redundant and is dropped. The *naive* pruner re-runs the full observation routine for every candidate vertex. The *fort-based* pruner exploits the connection between PDS and forts: when a removal fails, the unobserved remainder is a fort, and any vertex that is the sole solution vertex covering a known fort can be locked without re-running the full observation routine, saving the expensive check.

B. Experimental Setup

We evaluate on the ten BCSPWR graphs (BCSPWR01–10) from the SuiteSparse Matrix Collection, ranging from 39 to 5300 vertices. All heuristics use pure greedy selection (the single best vertex at each step).

C. Construction Quality Varies Widely

Raw solution sizes differ by an order of magnitude across constructors. On BCSPWR10 (5300 vertices), raw sizes range from 366 (H2) to 4465 (H3), with H1, H4, and H5 at 2275, 442, and 2307 respectively. The multi-hop and twin-guided heuristics over-reward high-degree hubs and structural redundancy, producing heavily bloated sets.

D. Pruning Narrows the Gap

After pruning, all five constructors converge to a narrow band. On BCSPWR10 the pruned sizes fall between 327 and 343—a spread of under 5%, despite raw sizes differing by more than a factor of twelve. This pattern holds across the benchmark: the pruning stage, rather than the construction strategy, determines the final solution size.

E. Effective-Degree-Based Greedy is Best End-to-End

H2 produces the least raw redundancy and the smallest pruned set on most graphs (327 on BCSPWR10, the minimum across all methods). It is the strongest choice both as a standalone constructor and as input to pruning. Table I summarizes raw and pruned sizes on representative instances.

TABLE I
RAW AND PRUNED PDS SIZES (SELECTED INSTANCES)

Dataset	n	H1	H2	H3	H4	H5
BCSPWR01	39	14/5	6/5	16/6	8/5	10/5
BCSPWR03	118	32/9	10/10	70/9	11/8	37/9
BCSPWR05	443	191/53	54/48	367/65	67/49	206/55
BCSPWR07	1612	686/174	187/167	1368/245	219/174	733/178
BCSPWR09	1723	519/174	188/173	1523/177	233/177	573/174
BCSPWR10	5300	2275/335	366/327	4465/338	442/343	2307/335

Entries are raw/pruned solution sizes.

F. Pruning Cost: Naive vs. Fort-Based

Both pruners reach identically sized minimal solutions, but they differ in computational cost. The naive pruner re-runs the full observation routine for every candidate vertex, whereas the fort-based pruner skips many of these checks by reusing cached forts. Table II reports wall-clock pruning time on each graph, with the speedup most pronounced on the larger instances BCSPWR06–10.

TABLE II
PRUNING TIME: NAIVE VS. FORT-BASED (H2)

Dataset	Naive (s)	Fort (s)	Speedup
BCSPWR06	1.0271	0.5105	2.01×
BCSPWR07	1.1282	0.6027	1.87×
BCSPWR08	1.3328	0.6689	1.99×
BCSPWR09	1.5879	0.8891	1.79×
BCSPWR10	10.2393	5.5567	1.84×

H2 (Effective-Degree-Based Greedy) timings.

G. Discussion

For PDS on power-network graphs, the choice of greedy construction rule strongly influences raw solution size, but a pruning stage is essential to reach minimal sets. Construction heuristics that appear far weaker by raw size—including the multi-hop and twin-guided variants—become nearly indistinguishable after pruning. The near-twin-guided heuristic in particular shows that near-twin structure is a weak signal for PDS, because observation spread is governed by

propagation reach rather than local neighborhood overlap; on graphs richer in near twin structure the picture may differ. Among constructors, effective-degree greedy introduces the least redundancy and yields the smallest final sets. The fort-based pruner attains the same minimal sizes as the naive pruner at reduced cost, which becomes significant on the largest grids.

III. CONCLUSION

We presented and evaluated a family of greedy heuristics for the Power Dominating Set problem, combined with naive and fort-based pruning, on the BCSPWR benchmark series. The key empirical finding is that pruning greatly improves solution quality across all constructors, with effective-degree-based greedy producing the smallest sets both before and after pruning. Fort-based pruning achieves the same solution quality as the naive pruner while reducing computational cost. Future work includes grid-specific reduction rules that exploit power-network topology directly.

REFERENCES

- [1] D. J. Brueni and L. S. Heath, “The pmu placement problem,” *SIAM Journal on Discrete Mathematics*, vol. 19, no. 3, pp. 744–761, 2005.
- [2] T. W. Haynes, S. M. Hedetniemi, S. T. Hedetniemi, and M. A. Henning, “Domination in graphs applied to electric power networks,” *SIAM journal on discrete mathematics*, vol. 15, no. 4, pp. 519–529, 2002.
- [3] A. Aazami, “Hardness results and approximation algorithms for some problems on graphs,” 2008.
- [4] D. Binkle-Raible and H. Fernau, “An exact exponential time algorithm for power dominating set,” *Algorithmica*, vol. 63, no. 1, pp. 323–346, 2012.
- [5] T. Bläsius and M. Göttlicher, “An efficient algorithm for power dominating set,” *Algorithmica*, vol. 87, no. 3, pp. 344–376, 2025.
- [6] V.-k. Tran and H.-s. Zhang, “Optimal pmu placement using modified greedy algorithm,” *Journal of Control, Automation and Electrical Systems*, vol. 29, no. 1, pp. 99–109, 2018.